Available online at www.sciencedirect.com

ScienceDirect

IFAC PapersOnline CONFERENCE PAPER ARCHIVE

IFAC PapersOnLine 55-4 (2022) 109–114

Utilizability of Navigation2/ROS2 in Highly Automated and Distributed Multi-Robotic Systems for Industrial Facilities ⋆

Tomas Horelican *

* *Central European Institute of Technology, Brno University of Technology, Brno, 612 00, Czech Republic*
Department of Control and Instrumentation, Faculty of Electrical Engineering and Communication, Brno University of Technology, Brno, 616 00, Czech Republic
(e-mail: Tomas.Horelican@vut.cz)

Abstract: The field of mobile robotics is a rapidly evolving space with a constant stream of new advancements. As the available technology matures and new features get accessible for lower prices, the scope of applicability for autonomous mobile robots naturally expands. Apart from simple home appliances, a particular area that currently attracts a surge of transformative research is that of industrial facilities and intelligent factories. A higher emphasis is gradually being placed on autonomous operations, where mobile robots start being suitable complements to human workers. Such an application, however, requires complex mechanisms to be able to handle even simple tasks, such as transferring cargo from one place to another while navigating a changing environment and avoiding dynamic obstacles. Robust localization and navigation solutions utilizing the most effective algorithms are necessary for such long and continuous operations that require a high degree of reliability and low error rates. *Robot Operating System* (ROS) is a highly flexible open-source framework enjoying wide adoption over the years. Its next-generation implementation (ROS2), together with the `Navigation2` project (created as a successor to the original *ROS Navigation Stack*), provide substantial conceptual enhancements with a strong focus on reliability, security, and performance, promising better compatibility with numerous sensor types, higher modularity, and a more controllable deterministic behavior. This paper will guide the reader through necessary steps to deploy the `Navigation2` stack onto an existing custom-built ROS2-operating robotic test-bed platform, serving as a concept for future research.

Keywords: ROS2, Navigation2, mobile robot, smart factory, multi-robot system, autonomous navigation, dynamic environment

1. INTRODUCTION

Mobile vehicles capable of independent and self-sustained locomotion have been a topic of research for well over several decades. By now this area comprises a vast range of concepts and mechanisms that can be assumed to fall into the general category of mobile robots. Starting from, by today's standards, simple laser-guided missile systems to autonomously driving commercial cars and complex anthropomorphic humanoids or animal-mimicking quadrupedal robots. The variety of tasks that can, or are expected to, be performed by mobile robots is also ever increasing, and a complete overview would be far out of sight of this article. Extreme environment (or extraterrestrial) exploration, medical assistance, customer service, or site surveillance are just some examples.

Novel approaches, such as multiple modality sensor data fusion, *Simultaneous Localization and Mapping* (SLAM) algorithms, and optimized *Artificial Intelligence* (AI) techniques further contribute to the acceleration of transition from manual teleoperation to fully decentralized autonomy for mobile robots, as was already investigated by Tripicchio et al. (2014). Greater availability of legged robots in recent years has sparked a new precedent for extreme environment traverse, reinforced by machine learning approaches to the problem of diverse and unpredictable terrain adaptation, which was successfully demonstrated by Kumar et al. (2021). In order to be efficient in their usage scenarios, the correct design choices have to be meticulously made in all of the major concerning fields,

⋆ The completion of this paper was made possible by the grant No. FEKT-S-20-6205 - "Research in Automation, Cybernetics and Artificial Intelligence within Industry 4.0" financially supported by the Internal science fund of Brno University of Technology. The work was supported by the infrastructure of RICAIP that has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 857306 and from Ministry of Education, Youth and Sports under OP RDE grant agreement No CZ.02.1.01/0.0/0.0/17_043/0010085.

10.1016/j.ifacol.2022.06.018

being: *locomotion*, *perception*, *cognition* and *navigation*, which is summarized by Rubio et al. (2019).

As the performance of robots in the four aforementioned fields of mobile robotics gets perfected, a paradigm of multiple robots cooperating in order to fulfill the given task emerges. Dubenko et al. (2019) propose a complete heterogeneous robotic system designed for monitoring the state of complex infrastructure facilities, while a cooperative aerial-ground multi-robot system for automated construction is explored by Krizmancic et al. (2020). Research into multi-robot cooperation is also being actively carried out by the group of Robotics and Artificial Intelligence at Brno University of Technology, where an aerial-ground heterogeneous robotic system is efficiently exploited for accurate mapping of ionizing radiation sources, as was demonstrated by Gabrlik et al. (2021) and Lazna et al. (2018).

A particular distinction that is gaining large traction in recent years is also, as highlighted by Fragapane et al. (2021), the transition from automated warehouse vehicles, which autonomously perform specific fixed tasks and navigate along predefined trajectories with a human operator oversight, to fully autonomous mobile robots capable of independent decentralized reasoning and efficient adaptation to dynamically changing environments. No longer being reliant on human supervision, these robots can effectively complement (or completely take over in some cases) the tasks performed by human workers. As evidenced from research by Baboli et al. (2015), new challenges emerge from incorporating a higher level of autonomy, such as the need for reliable and robust indoor localization solutions. Chik et al. (2016) show another important requirement, touching on the necessity of social-context-aware navigation architectures, and Capelli et al. (2019) present a promising approach attuned to multi-robot and human interactions through analyzing the nature of their dynamics. General requirements for industrial mobile robots along with the need for proper safety standards were reviewed by Shneier and Bostelman (2015).

The following sections of this article will introduce the ROS2 framework and the new *ROS 2 Navigation* system. A couple of related works concerning the deployment and utilization of ROS will be briefly presented, moving on to the implementation executed within this article. Afterwards, some practical results of a real-world demonstration as well as simulation run scenarios will be shown. The conclusion section will summarize the current achievements, and further prospective research along with its next natural direction, stemming from this demonstration, will be outlined.

1.1 Robot Operating System

The ROS project started out as an attempt to unify the scattered robotics development ecosystem, providing an intuitive general architecture (initially aimed at service robots) that could be universally adapted to any large-scale robotics project. It quickly gained traction among researchers and developers alike, as it enabled the various different implementations in all key robotics areas to be easily brought together into one ecosystem. Due to the open source nature of the project, its scale and relevance had grown exponentially, as seen by Boren and Cousins (2011).

It is not a functional operating system in and of itself as the name might suggest, but rather a higher layer, providing a universal communication interface (mostly through the IP stack) for different parts of the robotic system. Its distinctive attribute is the concept of *nodes*, working on top of the core operating system. The purpose of these *nodes* is arbitrary, from providing data packets from sensors to performing of complicated algorithms for high-level tasks. They pass *messages* in a prespecified format between each other through channels, which are called *topics*. Any *node* can be made to *subscribe* or *publish* to an arbitrary number of *topics*, which ensures a direct peer-to-peer architecture. A complete description of the system is provided by the authors Quigley et al. (2009).

The next generation of ROS aims to improve upon the shortcomings of the original, namely in the security and performance aspects, assuring safe and reliable operations for industrial robots, and real-time capabilities. This is also enabled by the adoption of the *Data Distribution Service* (DDS) scheme as a communication middleware (more closely analyzed by Maruyama et al. (2016)). From a networking perspective, the original implementation still relied on a centralized *master discovery node* to provide the correct initial connections between any given *nodes*. ROS2 no longer relies on such a master, leveraging the full nature of distributed peer-to-peer communication. Another distinct enhancement is the transition to so called *managed nodes*, which have deterministically controllable life-cycle states.

1.2 `Navigation2` (ROS 2 Navigation)

The project aims to redesign the concepts of the original *ROS Navigation Stack*, embracing the architectural improvements of ROS2, in order to make the framework fully equipped for the next generation of robotic systems. Utilizing *Behavior Trees* (BT) instead of *Finite State Machines* (FSM) for state estimations and transitions makes it more suitable for complicated real-world tasks. A strong focus on multi-core processing adheres to the real-time faculties of ROS2. The new framework promises better performance in massively dynamic environments and support for a wider selection of robot shapes, locomotion types, and high-level algorithms, with an emphasis on high modularity.

The well known concept of *layered costmaps*, described already by Lu et al. (2014), is widely utilized for world representation. This method permits more efficient fusion of map data with various distinct characteristics and high customizability of the complete map hierarchy. The full map representation of the world can be, ergo, constructed with several independent layers, each updated by their corresponding data source. The most typical types widely used in *Navigation2* are: the *inflation*, *static*, *voxel*, or *obstacle* layers, each serving its specific modality to complete the full *costmap* representation. A new addition to this hierarchy, introduced by Macenski et al. (2020b), is the so-called *Spatio-Temporal Voxel Layer* (STVL). Attuned particularly to dynamic environments, it provides a 3D world representation that can be built from sensors such

as lidars, laser scanners, or depth cameras, with an ability to decay over time as the data from sensors becomes rarer.

ROS2 and `Navigation2` incorporate the *SLAM Toolbox*, presented by Macenski and Jambrecic (2021), as the main package for SLAM solutions. It can provide real-time mapping capabilities for large areas thanks to state-of-the-art graph-based implementations, which makes it well suited for dynamic environments. Focusing on customizability, it also provides many options to match specific use-case requirements. Keeping up with the latest trends, the developers of `Navigation2` express strong interest in supporting latest state-of-the-art *Visual SLAM* (VSLAM) approaches (as evidenced by Merzlyakov and Macenski (2021)) that might replace traditional techniques, which require expensive lidar sensors. With the costs of modern RGB-D and stereo cameras rapidly decreasing and AI-based visual processing algorithms being ever more effective, these techniques might present the next revolution in the SLAM field.

The new framework also continues to provide strong support for well known advanced localization techniques based on particle filter algorithms, such as described by Fox (2001).

In their demonstrative article, Macenski et al. (2020a) provide a more comprehensive description and demonstration of the project's capabilities, taking into account insights from the brand new ROS2 framework.

2. RELATED WORK

Hu et al. (2015) propose a solution for multi-robot system operations with a custom ROS-based architecture. They were able to demonstrate efficient real-time performance with multiple robots, however their research is based on the first generation of ROS, which might transpire to be outdated when the many enhancements of ROS2 are taken into account.

Zheng (2017) already provides an extensive deep dive into the configuration and deployment of the *ROS Navigation Stack*. Due to the substantial changes in ROS2, some ideas might not apply the same way anymore. This article, however, is not aimed to be a full-featured substitution to the aforementioned paper, but more of a proof-of-concept demonstration for future state-of-the-art research.

3. IMPLEMENTATION

The main objective of this article is not to demonstrate the full robotic system with all of the features and competencies, but rather to bring attention to a particular constituent, without which the overall high-level behavior would not be realized. However it would be considered incomplete without at least a principal description of the base test-bed robotic platform, onto which the implementation presented herein was applied.

3.1 Base platform

The navigation layer configuration was composed for a simple differential-drive chassis robot (see Fig. 1) with a single 32-channel *Velodyne HDL-32E* lidar sensor, used

Fig. 1. Loki-1: A demonstrative test-bed platform for the initial concept realization.

as the primary source of data for real-time obstacle detection and avoidance. High precision encoders satisfied `Navigation2` requirements for odometry. Its overall dimensions are 40 x 35.5 x 52 cm, including the lidar on top. The system was equipped with a *Raspberry Pi 4B* mini-computer, which handled all of the low-level operations necessary to control the chassis, and provided odometry data and geometric transformations required to represent the physical robot model with all of its constituents in 3D space. Each one of these data sources adheres to the ROS standards for message formats, which allows for fluid integration of the navigation layer and seamless visualization in the RViz environment. All of the high-level computations, which are part of the navigation layer, were carried out by a high-performance *11th-Gen Intel NUC*, also a component of the robotic system. Both computers were using an *Ubuntu 20.04* OS. Internal communication was handled through a *Mikrotik* router, which was passing the ROS messages between individual devices. The platform can also be equipped with additional sensors, which are, however, not relevant to this article. It was developed by the group of Robotics and Artificial Intelligence at Brno University of Technology.

3.2 `Navigation2` setup and deployment

The whole framework provides a vast range of configuration options and a full detailed exploration would not be feasible within just six pages. This article will instead attempt to explain the most relevant or distinct ones, while providing a sufficient general overview. The intention was to leave as much as possible in the default configuration to explore rapid deployability on an arbitrary platform. Parameters, which required custom tuning or resulted in more desirable behavior will be highlighted. ROS2 version code-named *foxy* was used throughout the entirety of this article.

The operational architecture of `Navigation2` comprises several *lifecycle nodes*, which are all independently configurable through a `*.yaml` file, and necessary for the full system to work. Each *node* anchors a server, which can host various task-specific implementations defined as *plugins*. Table 1 summarizes all of the primary *plugins*, utilized for this article.

General parameters, such as `robot base frame` have to correspond to actual values published by the particular robot (in this case, it was always `loki_1_base_link`). The `global frame` was always set according to the spe-

cific context (so `loki_1_odom` for the `local costmap` and `recoveries` servers, and `map` for the `global costmap` and `bt navigator` servers). The `odom topic` values were, naturally, set to their corresponding topic names.

Top-level behavior is orchestrated by the `bt navigator` *node*, which works with a *Behavior Tree* representation stored in an `xml` format. The default

- `navigate_w_replanning_and_recovery.xml`

(its logical structure is the same as was used by Macenski et al. (2020a)) was mildly modified with different recovery behaviors to:

(1) Clear all costmaps.
(2) Wait for 5 seconds.
(3) Perform 2 opposing $\pm\pi/4$ *rad* ($\pm$45 deg) spins.
(4) Attempt to slowly back-up by 5 cm.

The corresponding `Wait, Spin,` and `BackUp` *plugins* had to also be defined in the `recoveries` server section of the configuration `yaml` file. Similarly, all plugins that are launched form the `xml` BT have to be defined in the `bt navigator` section, which for this article constituted the default setup. Unexpected navigation failures were observed with default server timeout settings, therefore all calls in the BT were individually raised to a 200 ms timeout.

Navigation as a whole is handled by the `waypoint follower`, and the `controller`, `planner`, and `recoveries` servers. The `waypoint follower` secures general motion through goals and it did not require any changes beyond defaults. All fall-back behavior (in case no valid paths can be found) is managed by the `recoveries` server, which operated at 10 Hz and was configured to use actions mentioned above.

A global trajectory plan is managed by the `planner` server, which used the default grid-based `NavfnPlanner` *plugin* utilizing the *Dijkstra* algorithm, with a planning frequency of 10 Hz. A goal tolerance of 0.2 m was set. The `allow unknown` parameter often resulted in planning straight through a newly observed obstacle even when correctly recognized at first, and disabling this option seemed to remedy the undesired behavior.

A local trajectory plan is maintained by the `controller`, which involved most of the customizations. This and the `recoveries` servers are the only ones that have direct control over the robot chassis. It utilizes multiple *plugins*, configured as follows. The `SimpleProgressChecker`, with a minimum required movement distance of 0.1 m in at least 10 s, was verifying that the robot was not being stuck in a stationary state. The `SimpleGoalChecker` made sure the robot was correctly positioned at the desired goal location within the default tolerances (0.25 m and 0.25 rad). `DWBLocalPlanner` was used as the main *plugin* for local trajectory corrections. It generates multiple possible trajectories and dynamically prioritizes the best one based on a set of *critics*, which for this article were: `RotateToGoal`, `Oscillation`, `BaseObstacle`, `GoalAlign`, `PathAlign`, `PathDist`, and `GoalDist`. Their weights were set to prioritize for global path coincidence and obstacle avoidance (in the same scale) over goal proximity. The `RotateToGoal slowing factor` was set to slow the robot's motion by half at goal proximity. Kinematic parameters require individual tuning for each robot platform. The most important ones for this paper were: minima and maxima of velocities in the x axis, xy translational speeds, and angular velocities, which were all set to $v \in [0, 0.3]$ m/s, and $\omega \in [0, 1.5]$ rad/s, respectively. Acceleration and deceleration limits in the x axis were set to ± 2.5 m/s^2 and their angular counterparts to ± 3.2 rad/s^2. All else was left in the default state.

Table 1. Main `Navigation2` plugins used

Server	Costmaps	Recoveries	Controller
Plugins	StaticLayer	Wait	SimpleProgressChecker
	InflationLayer	Spin	SimpleGoalChecker
	ObstacleLayer	BackUp	DWBLocalPlanner
	VoxelLayer		
Server	Planner		
Plugins	NavfnPlanner		

Localization was, as of writing of this paper, mainly accomplished through an externally provided static map by the `map` server, utilizing the differential chassis odometry as a main source of position data and a *Motion Capture* (MoCap) setup for pose drift corrections, using the *Extended Kalman Filter* (EKF) algorithm. For this, ROS already provides a ready-to-use package called `robot_localization`. The static maps were created manually from detailed measurements and ground plans with image editing software. Future implementations should utilize the AMCL localization package for truly self-sustained and independent robot pose estimation adapted for arbitrary areas. It will also be desirable to employ the *SLAM Toolbox* as a more efficient map creation method, and to get on top of state-of-the-art instrumentations.

The global costmap was set to publish and update its data at a rate of 1 Hz with a resolution of 0.05 m/pixel, and was composed from three layers. The robot's footprint was set by the `robot radius` parameter to 0.35 m, which is more than the largest distance from center on the real chassis by about 0.8 m to promote safer planning in dangerous zones. The `StaticLayer` *plugin* was set to subscribe to map updates. It was found that the server would automatically adjust the static costmap layer resolution to match the source map regardless of the `resolution` parameter, therefore using high-resolution maps was causing unnecessary bandwidth overload on the whole system. The `InflationLayer` *plugin* was configured with an `inflation radius` of 0.55 m and the `cost scaling factor` was lowered to 5 in order to promote a gentler exponential decay curve across the inflation radius, blowing out the obstacles more. The `inflate around unknown` parameter was enabled. The `ObstacleLayer` *plugin* was subscribing to the corresponding single source of `PointCloud2` data published by the lidar. The `min` and `max obstacle height` values were set 0.1 m and 0.6 m to compensate for ground noise and also reflect the robot height. The `expected update rate` on the point-cloud topic was set to 0.2 s, which was about half of the true publishing rate from the lidar, and `observation persistence` was raised to 0.1 s.

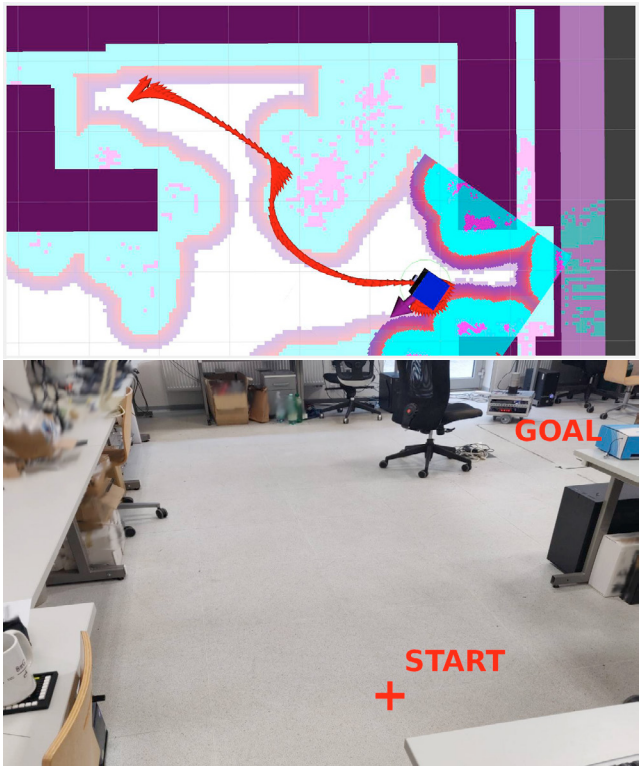

Fig. 2. Dynamic planning in a real-world environment. *Top*: odometry and costmap data in RViz. *Bottom*: view of the scenario setting.

Local costmap update frequency was 5 Hz and it was publishing its data at 2 Hz with a resolution of 0.025 m/pixel. Its size was set to 3 x 3 m and the `rolling window` option was enabled. It utilized the `InflationLayer` *plugin* with the same parameters as the global costmap and a `VoxelLayer`, which also subscribed to the same single lidar topic containing `PointCloud2` data. The voxel `z resolution` was 0.05 m and `mark threshold` was set so that only 3 or more voxels on top of each other can mark a costmap cell as occupied.

Deployment to the robot platform was arranged through `systemd` services. Custom *python launch files* were used to start-up all the necessary *nodes* with the required parameters. It is important to note that the *navigation* service has to be launched after all of the lidar velodyne *nodes* have been started, otherwise time synchronization issues with frame-dropping were occurring.

4. RESULTS

A dynamic scenario is shown in Figure 2, where the robot first started with a straight path towards the set goal (marked with a purple arrow). A foreign object was placed right into the robot's path as it was passing through the middle portion, forcing it to dynamically re-plan its trajectory (as shown by the red odometry arrows). As can be seen, the robot was able to plan and follow an initial valid trajectory, then successfully react to the changing environment and divert its original course, while still reaching the accessible initial goal pose.

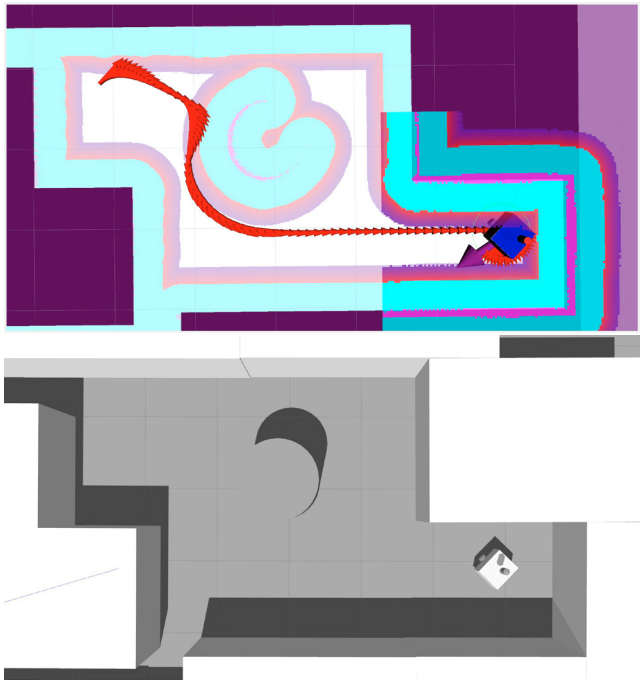

Fig. 3. Simulating the real-world scenario in a virtual environment. *Top*: odometry and costmap data in RViz. *Bottom*: 3D world model in Gazebo.

4.1 Simulation

The same `Navigation2` configuration could also be rapidly tested in a virtual *Gazebo* representation simulating the real-world, as the message formats and robot representation are in conformity with the real physical case. Figure 3 shows the same scenario as before, realized in a virtual environment. The same obstacle-evading behavior was observed when a cylinder object had been dynamically placed in the robot's path.

5. CONCLUSION

This article showcased the capabilities of latest available technologies for robotic applications. The ROS2 and `Navigation2` frameworks can easily be adapted to a specific custom robot design and, as can be seen, even the simplest configuration possible already yields promising results.

Further research should be focused on fully utilizing `Navigation2` localization capabilities (with the AMCL and SLAM packages) and more effective *Behavior Tree* structures customized for safe and reliable industrial operations. More attention should be placed on fine-tuning the available parameters and utilizing the costmap hierarchy efficiently for truly reliable multi-robot behavior in a dynamic human-interacting area. Investigations into more advanced navigation *plugins* (such as the *Regulated Pure Pursuit* `controller` and *Hybrid-A** `planner` algorithm for example), or perhaps new methods altogether, should be performed.

REFERENCES

Baboli, A., Okamoto, J., Tsuzuki, M.S., Martins, T.C., Miyagi, P.E., and Junqueira, F. (2015). Intelli-

gent manufacturing system configuration and optimization considering mobile robots, multi-functional machines and human operators: New facilities and challenge for industrial engineering. *IFAC-PapersOnLine*, 48(3), 1912–1917. doi:https://doi.org/10.1016/j.ifacol.2015.06.366. URL https://www.sciencedirect.com/science/article/pii/S2405896315006059. 15th IFAC Symposium on Information Control Problems in Manufacturing.

Boren, J. and Cousins, S. (2011). Exponential growth of ros [ros topics]. *IEEE Robotics Automation Magazine*, 18(1), 19–20. doi:10.1109/MRA.2010.940147.

Capelli, B., Secchi, C., and Sabattini, L. (2019). Communication through motion: Legibility of multi-robot systems. In *2019 International Symposium on Multi-Robot and Multi-Agent Systems (MRS)*, 126–132. doi:10.1109/MRS.2019.8901100.

Chik, S.F., Yeong, C.F., Su, E.L.M., Lim, T.Y., Subramaniam, Y., and Chin, P.J.H. (2016). A review of social-aware navigation frameworks for service robot in dynamic human environments. *Journal of Telecommunication, Electronic and Computer Engineering (JTEC)*, 8(11), 41–50. URL https://jtec.utem.edu.my/jtec/article/view/1408.

Dubenko, Y., Gura, D., and Dyshkant, E. (2019). Monitoring complex infrastructure facilities state using mobile robots: Problem analysis. In *2019 International Multi-Conference on Industrial Engineering and Modern Technologies (FarEastCon)*, 1–6. doi:10.1109/FarEastCon.2019.8934179.

Fox, D. (2001). Kld-sampling: Adaptive particle filters and mobile robot localization. *Advances in Neural Information Processing Systems (NIPS)*, 14(1), 26–32.

Fragapane, G., de Koster, R., Sgarbossa, F., and Strandhagen, J.O. (2021). Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda. *European Journal of Operational Research*, 294(2), 405–426. doi:https://doi.org/10.1016/j.ejor.2021.01.019. URL https://www.sciencedirect.com/science/article/pii/S0377221721000217.

Gabrlik, P., Lazna, T., Jilek, T., Sladek, P., and Zalud, L. (2021). An automated heterogeneous robotic system for radiation surveys: Design and field testing. *Journal of Field Robotics*, 38(5), 657–683. doi:https://doi.org/10.1002/rob.22010. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/rob.22010.

Hu, C., Hu, C., He, D., and Gu, Q. (2015). A new ros-based hybrid architecture for heterogeneous multi-robot systems. In *The 27th Chinese Control and Decision Conference (2015 CCDC)*, 4721–4726. doi:10.1109/CCDC.2015.7162759.

Krizmancic, M., Arbanas, B., Petrovic, T., Petric, F., and Bogdan, S. (2020). Cooperative aerial-ground multi-robot system for automated construction tasks. *IEEE Robotics and Automation Letters*, 5(2), 798–805. doi:10.1109/LRA.2020.2965855.

Kumar, A., Fu, Z., Pathak, D., and Malik, J. (2021). RMA: rapid motor adaptation for legged robots. *CoRR*, abs/2107.04034. URL https://arxiv.org/abs/2107.04034.

Lazna, T., Gabrlik, P., Jilek, T., and Zalud, L. (2018). Cooperation between an unmanned aerial vehicle and an unmanned ground vehicle in highly accurate localization of gamma radiation hotspots. *International Journal of Advanced Robotic Systems*, 15(1), 1729881417750787. doi:10.1177/1729881417750787. URL https://doi.org/10.1177/1729881417750787.

Lu, D.V., Hershberger, D., and Smart, W.D. (2014). Layered costmaps for context-sensitive navigation. In *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 709–715. doi:10.1109/IROS.2014.6942636.

Macenski, S. and Jambrecic, I. (2021). Slam toolbox: Slam for the dynamic world. *Journal of Open Source Software*, 6(61), 2783. doi:10.21105/joss.02783. URL https://doi.org/10.21105/joss.02783.

Macenski, S., Martín, F., White, R., and Clavero, J.G. (2020a). The marathon 2: A navigation system. *CoRR*, abs/2003.00368. URL https://arxiv.org/abs/2003.00368.

Macenski, S., Tsai, D., and Feinberg, M. (2020b). Spatio-temporal voxel layer: A view on robot perception for the dynamic world. *International Journal of Advanced Robotic Systems*, 17(2), 1729881420910530. doi:10.1177/1729881420910530. URL https://doi.org/10.1177/1729881420910530.

Maruyama, Y., Kato, S., and Azumi, T. (2016). Exploring the performance of ros2. In *Proceedings of the 13th International Conference on Embedded Software*, EMSOFT '16. Association for Computing Machinery, New York, NY, USA. doi:10.1145/2968478.2968502. URL https://doi.org/10.1145/2968478.2968502.

Merzlyakov, A. and Macenski, S. (2021). A comparison of modern general-purpose visual SLAM approaches. *CoRR*, abs/2107.07589. URL https://arxiv.org/abs/2107.07589.

Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., Ng, A.Y., et al. (2009). Ros: an open-source robot operating system. In *ICRA workshop on open source software*, volume 3, 5. Kobe, Japan.

Rubio, F., Valero, F., and Llopis-Albert, C. (2019). A review of mobile robots: Concepts, methods, theoretical framework, and applications. *International Journal of Advanced Robotic Systems*, 16(2), 1729881419839596. doi:10.1177/1729881419839596. URL https://doi.org/10.1177/1729881419839596.

Shneier, M. and Bostelman, R. (2015). Literature review of mobile robots for manufacturing. doi:https://doi.org/10.6028/NIST.IR.8022. URL https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=915942.

Tripicchio, P., Satler, M., Avizzano, C., and Bergamasco, M. (2014). Autonomous navigation of mobile robots: From basic sensing to problem solving. *20th IMEKO TC4 Symposium on Measurements of Electrical Quantities: Research on Electrical and Electronic Measurement for the Economic Upturn, Together with 18th TC4 International Workshop on ADC and DCA Modeling and Testing, IWADC 2014*, 1–6. URL https://www.imeko.org/publications/tc4-2014/IMEKO-TC4-2014-387.pdf.

Zheng, K. (2017). ROS navigation tuning guide. *CoRR*, abs/1706.09068. URL http://arxiv.org/abs/1706.09068.